AutoML CI/CD/CT Capstone Project Final Report

Sepehr Heydarian, Archer Liu, Elshaday Yoseph, Tien Nguyen

Table of contents

1 Executive Summary

2 Introduction

Wildfires represent a significant and growing environmental threat, with the potential to cause widespread ecological and economic damage. Defence Research and Development Canada (Defence Research and Development Canada (2022)) emphasizes the importance of early detection in mitigating the impact and spread of these events.

Our Capstone partner, Bayes Studio, is a Vancouver-based startup developing advanced AI-driven tools for environmental monitoring and wildfire detection. They deploy specialized camera systems equipped with lightweight models that operate as edge devices, allowing for real-time, on-site inference.

At the outset of the project, Bayes Studio's workflow for updating their wildfire detection models involved several manual steps: labeling new environmental imagery, fine-tuning a base object detection model, and compressing it for edge deployment. This process was time-consuming and constrained their ability to quickly respond to changing conditions in wildfire imagery.

The available data consisted of open-source image datasets labeled across five wildfire-relevant classes. These images served as the foundation for developing and evaluating the pipeline components.

Over the course of the project, we refined the original problem statement into three concrete objectives:

1. **Semi-automated labeling**: To accelerate data annotation, we implemented a semi-automated labeling approach supported by a human-in-the-loop process. This ensures that uncertain or edge cases are reviewed and corrected by human annotators while reducing the overall labeling effort.
2. **Model fine-tuning**: We used the YOLOv8 object detection model as a base and fine-tuned it on the newly labeled data, allowing the model to adapt to updated image distributions and maintain detection performance over time.
3. **Model compression**: To meet the hardware constraints of Bayes Studio's edge devices, we applied model compression techniques, including knowledge distillation and quantization.

These objectives were designed to address key operational challenges faced by Bayes Studio. By reducing manual effort in labeling, streamlining model fine-tuning, and supporting hardware-efficient deployment, the system enables faster iteration and improved version control over detection models. This provides the partner with a more scalable and maintainable workflow for real-time wildfire detection.

3 Data - Pipeline Input

The input to the pipeline consists of image files in standard formats such as `.jpg`, paired with YOLO-formatted `.txt` label files. Each label file corresponds to a single image and contains one line per object instance. These lines specify the object's class ID and normalized bounding box coordinates in the following format: `x_center`, `y_center`, `width`, `height`. For example: `1 0.757593 0.34663 0.06101 0.3655`.

This line represents an object of class ID 1 with a bounding box centered at (0.757593, 0.34663) and a normalized width and height of 0.06101 and 0.3655, respectively.

The object detection task focuses on five relevant classes: 1. Fire
2. Smoke
3. Lightning
4. Vehicle
5. Person

These categories were selected based on their relevance to early wildfire detection and response.

In terms of volume, the partner anticipates receiving approximately 500 new images per month. This stream of incoming data forms the basis for the pipeline's three core stages:
(1) semi-automated labeling supported by a dual-model system and human-in-the-loop review,
(2) data augmentation to increase class and image diversity, and
(3) re-training of the base detection model on the updated dataset.

This setup ensures that the model can continue to adapt to new environmental conditions and remain effective over time.

4 Data Science Methods and Product

Our AutoML pipeline is designed for wildfire object detection, with a focus on automation, accuracy, and efficiency. The system continuously processes new data and improves model performance through an iterative loop of labeling, training, distillation, and quantization. Below is a high-level summary of the core components:

4.1 End-to-End Pipeline Flow

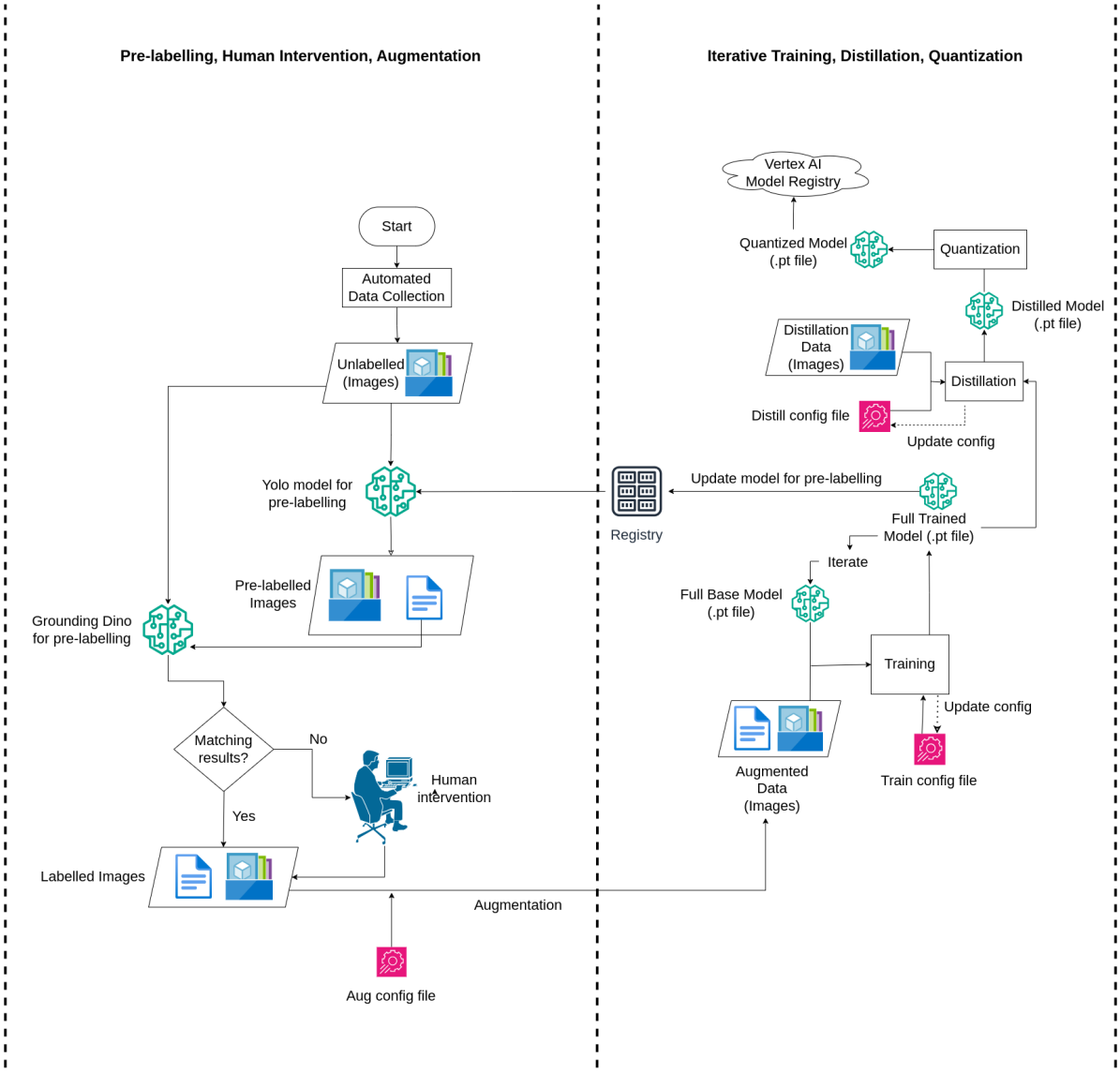

Figure 1: Pipeline Overview Diagram

4.2 Pipeline Stages

1. **Automated Labeling and Matching**
   Automatically generates object labels using two models (YOLOv8 and Grounding DINO). If both models agree, the label is accepted. Disagreements are routed for human review.

2. **Human-in-the-Loop**
   Mismatched cases are corrected through a Label Studio interface. This step ensures that there are no dropped mismatched images, thus enriching a high-quality training dataset.

3. **Augmentation**
   Labeled data is augmented using transformations such as flipping, rotation, and brightness changes. This step increases dataset diversity and robustness.

4. **Model Training**
   A YOLOv8 model is trained on the labeled and augmented dataset. Training is guided by configuration files that allow reproducible tuning and GPU/CPU flexibility.

5. **Model Distillation**
   The trained model is distilled into a smaller version for faster inference, reducing model size while preserving performance.

6. **Quantization**
   The distilled model is quantized to further reduce its size for edge device deployment.

Each component is explained in more detail in the following sections.

4.3 Automated Labeling and Matching

1. **Motivation**

   Manual annotation is time-consuming and prone to inconsistencies. To streamline this process, we employ a dual-model pre-labeling strategy using YOLOv8(Jocher, Chaurasia, and Qiu 2023) and Grounding DINO(Liu et al. 2023). This setup leverages model agreement to reduce manual labeling needs, while preserving uncertain cases for review. The goal is to automatically generate accurate labels with minimal human intervention.

2. **Input**

   - Unlabeled images placed in `automl_workspace/data_pipeline/input/`
   - Two pre-trained object detection models:
     - **YOLOv8**: Trained specifically on our five wildfire-related categories – `Fire, Smoke, Lightning, Human, Vehicle`.
     - **Grounding DINO**: Uses the same 5 categories as prompts to localize objects with open-vocabulary detection, since models like GroundingDINO rely on prompts to identify target objects.

3. **Workflow Steps**

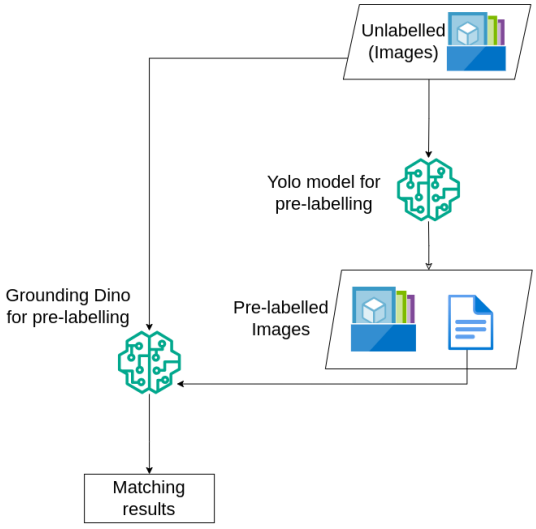

Figure 2: Labeling and Matching Workflow

- **Step 1:** YOLOv8 runs on the images and predicts bounding boxes and class labels
- **Step 2:** Grounding DINO runs on the same images using the same 5 categories as prompts (`Fire, Smoke, Lightning, Human, Vehicle`)
- **Step 3:** Our matching script compares predicted labels for each image from both models using Intersection over Union (IoU) thresholds:
  - If both models agree on the predicted labels, store the results in `automl_workspace/data_pipeline/labeled/`
  - If they disagree on any of the predicted labels, send files to `automl_workspace/data_pipeline/label_studio/pending/` for human review
- **Step 4:** Matched labels are saved in YOLO-compatible JSON format for augmentation and training

4. **Output**

- A curated dataset of confidently labeled images saved in `automl_workspace/data_pipeline/labeled/`
- Uncertain cases routed to the human-in-the-loop step for correction
- Each labeled file includes bounding box coordinates, class name, and confidence score

5. **Impact**

This automated labeling workflow reduces reliance on full manual annotation, ensures consistent labeling across large batches of images, leading to faster dataset preparation while maintaining label accuracy.

4.4 Human-in-the-loop

1. **Motivation**

   A **high-performing model** relies on a **high-quality labeled dataset**. While YOLOv8 and GroundingDINO agree on many predictions, there are still **mismatches** that need correction. Instead of discarding these potentially valuable samples, we introduce a **human-in-the-loop workflow** to verify and fix labeling errors with Label Studio(Heartex 2021). This process improves data quality with **minimal manual effort**, streamlining the correction process and reducing the need for manual file editing.

2. **Input**

   - JSON files with incorrect or uncertain predictions from YOLO, containing labels, bounding box coordinates, and confidence score
   - Corresponding image files referenced in those predictions

3. **Workflow Steps**

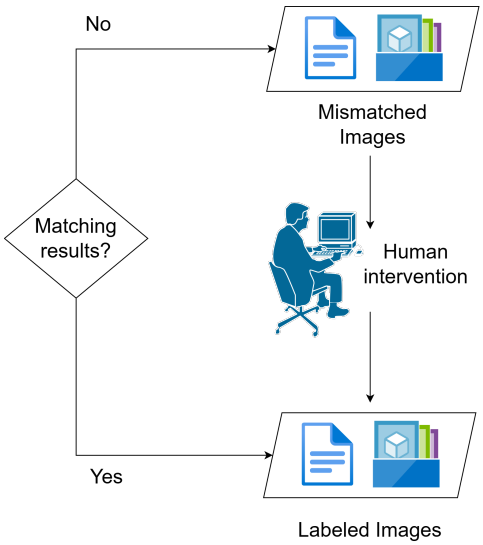

Figure 3: Human-in-the-loop workflow diagram

   - **Step 1:** Place mismatched prediction files in `automl_workspace/data_pipeline/label_studio/pending/`

- **Step 2:** Run the script to convert them into Label Studio tasks using images from `automl_workspace/data_pipeline/input/`
- **Step 3:** Review and fix labels visually in the intuitive Label Studio interface

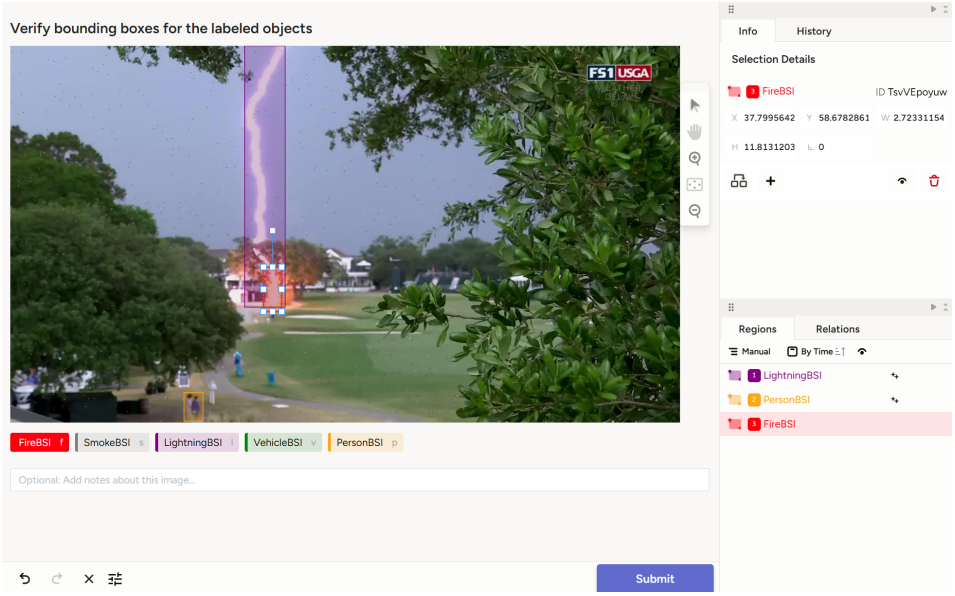

Figure 4: Label Studio Interface for Human Review

- **Step 4:** Export the reviewed results to `automl_workspace/data_pipeline/label_studio/results/`
- **Step 5:** Reviewed prediction files are moved from pending directory to `automl_workspace/data_pipeline/labeled/`, available for the training phase

The script automatically tracks review status throughout the process.

4. **Output**

   - Corrected label files (JSON), ready for training
   - The file names of Label Studio outputs are versioned with timestamps automatically for easy tracking

5. **Impact**

   The human-in-the-loop workflow makes better use of mismatched data. Reviewers can easily correct labels through an intuitive interface, without editing raw files manually. The reviewed results are saved in a consistent format, ready for training. Overall, this workflow improves the quality of the training dataset, allowing the model to learn from a wider range of examples.

5 Data Augmentation

5.1 Motivation

A single scene can look very different depending on external conditions such as lighting, weather, occlusion, and the angle or position of the camera. To help the model generalize better to these real-world variations, we apply data augmentation techniques to artificially increase the diversity of the training dataset. This process improves robustness and reduces the risk of overfitting, especially in low-data or imbalanced class scenarios.

In this project, the partner specifically requested that augmentations be applied prior to training, rather than using in-training augmentation. This decision was motivated by the need to minimize computational overhead and training time.

5.2 Input

The augmentation process uses labeled images along with their corresponding YOLO-formatted annotation files. These images are sourced from the matched outputs of the pre-labeling pipeline and are stored in the `labeled` directory. Augmentation behavior is controlled via the `augmentation_config.json` file, which allows customization of parameters such as the number of augmentations per image, transformation probabilities, and limits for effects like blur intensity.

5.3 Workflow

- **Step 1**: Load each image and its corresponding annotation.
- **Step 2**: For each image, generate a fixed number of augmented versions (e.g., 3).
- **Step 3**: Apply a set of transformations using the `Albumentations` library, including:
  - Horizontal flips
  - Brightness and contrast adjustments
  - Hue and saturation shifts
  - Gaussian blur (with a configurable limit)
  - Gaussian noise
  - Rotations

  - Conversion to grayscale

- **Step 4**: Update the bounding boxes to reflect the applied transformations. This is handled automatically by the `Albumentations` library.
- **Note**: Images without any objects of interest are excluded from augmentation.
- **Step 5**: Save the augmented images along with their updated annotations.

5.4 Output

The result is a larger and more varied labeled dataset prepared for training. This includes both the augmented images with associated annotation files and the original non-augmented images. Images without any predicted or labeled objects are stored separately in the `no_prediction_images` folder.

5.5 Impact

By augmenting the data prior to training, the model is exposed to a wider range of visual conditions without requiring additional manual data collection or labeling. This helps reduce overfitting and improves generalization performance, particularly under different lighting conditions, weather variations, and camera viewpoints.

6 Model Training & Fine Tuning

6.1 Motivation

Once labeled and augmented data is available, the detection model must be re-trained to incorporate new information. Continuous re-training enables the model to adapt to evolving wildfire conditions and the characteristics of newly collected data. The goal of this stage is to automate the fine-tuning of a base YOLOv8 model using the most recent dataset, minimizing the need for manual effort.

6.2 Input

The training stage uses the full set of labeled images, including both augmented and non-augmented examples. Images without any labeled objects are also included as true negatives. The model to be fine-tuned is selected from the model registry. By default, this is the most recently updated model yolo model from previous iteration of the pipeline, though users can also specify a custom path or fall back to an initial base model such as `nano_trained_model.pt`.

Training hyperparameters including settings like the number of epochs, learning rate, and input image size—are defined in the `train_config.json` file. This configuration supports any training argument compatible with the YOLOv8 framework.

6.3 Workflow

- **Step 1**: Load the training configuration and identify the most recent base model.
- **Step 2**: Organize the data into training, validation, and test splits.
- **Step 3**: Fine-tune the base YOLOv8 model using the prepared data and configuration settings.
  - Training is conducted using the Ultralytics YOLOv8 training API.
  - Performance metrics are logged during training.
- **Step 4**: Save the updated model in the model registry using a timestamped filename.
- **Step 5**: Store metadata, including training date, configuration settings, and performance results, in a corresponding metadata file.

6.4 Output

The training stage produces a fine-tuned YOLOv8 model that reflects the latest available data. The model is saved with a filename that follows the pattern `[YYYY-MM-DD]_[HH_MM]_updated_yolo.pt`, and is stored in the model registry. A metadata file with the same name records training details and performance metrics to support versioning and reproducibility.

6.5 Impact

This process enables regular fine-tuning as new data becomes available, helping ensure that the detection model remains accurate and up-to-date. The modular structure also allows users to experiment with different hyperparameters or revisit older models for retraining as needed. Over time, this approach supports continual improvement in model performance and maintainability.

6.6 Knowledge Distillation (Outline)

1. **Motivation**
   - Large models are computationally expensive for real-time wildfire detection
   - Knowledge distillation transfers expertise from a larger teacher model to a smaller student model
   - Enables faster inference while maintaining detection accuracy

2. **Input**

   - Teacher model: Pre-trained YOLOv8 model with proven wildfire detection capabilities
   - Student model: Smaller YOLOv8n architecture initialized with pretrained weights from COCO 80 classes.
   - Training dataset with 5 classes (FireBSI, LightningBSI, PersonBSI, SmokeBSI, VehicleBSI)
   - Configuration file specifying distillation parameters and hyperparameters

3. **Workflow Steps**

   - Step 1: Load and prepare both teacher and student models
   - Step 2: Freeze first 10 layers of student model (backbone) for stable feature extraction
   - Step 3: Train student model using combined loss function:
     - Detection loss (_detection = 1.0) for direct supervision
     - Distillation loss (_distillation = 2.0) for knowledge transfer:
       * Bounding box distillation (_dist_ciou = 1.0)
       * Classification distillation (_dist_kl = 2.0)
   - Step 4: Regular checkpointing and model state preservation
   - Step 5: Save final distilled model for downstream tasks.

4. **Output**

   - Distilled YOLOv8n model with reduced size and complexity
   - Training logs and metrics for performance analysis
   - Checkpoint files for training resumption
   - Final model saved in `mock_io/model_registry/distilled/latest/`

5. **Impact**

   - Reduced model size and computational requirements
   - Faster inference times suitable for edge deployment
   - Maintained detection accuracy through knowledge transfer
   - Enables efficient deployment on resource-constrained devices

6.7 Quantization

1. **Motivation**

   Even after model distillation, running inference on edge devices (e.g., drones, cameras) can still be too slow or resource-intensive. To address this, quantization is used to **reduce the size of the model** and **improve inference speed** by lowering the numerical precision of model weights. This makes the model more efficient and suitable for real-time wildfire detection in the field.

2. **Input**

   - The distilled YOLO model from the previous distillation step of the pipeline
   - (Optional) labeled images and JSON annotations for calibration, required for IMX quantization

3. **Workflow Steps**

   - **Step 1:** Set quantization method in `automl_workspace/config/quantize_config.json` (`"FP16"`, `"ONNX"`, or `"IMX"`)
   - **Step 2:** Based on method:
     - **FP16**: Converts weights to float16, fast and accurate, saved as `_fp16.pt`
     - **ONNX**: Converts to ONNX and applies dynamic quantization, saved as `_onnx_quantized.onnx`
     - **IMX**: Uses Ultralytics export with calibration data to create IMX-ready format (Linux only), saved as `_imx.onnx`
   - **Step 3:** Quantized model is saved in `automl_workspace/model_registry/quantized/`

4. **Output**

   - A smaller, faster model ready for deployment:
     - ~50% size reduction with FP16, minimal accuracy drop
     - ~75% size reduction with ONNX, slight accuracy tradeoff
     - IMX-ready model for Sony Raspberry Pi AI Camera

5. **Impact**

   The quantized model is lightweight enough to run directly on edge devices that have limited computing power. This meets the partner's need to deploy the model on Sony Raspberry Pi AI Cameras for real-time wildfire detection. Additionally, there are multiple quantization options (FP16, ONNX, IMX) available, making it easy to switch methods based on different deployment needs in the future.

7 Conclusions and Recommendations

7.1 Problem Recap

Our partner, Bayes Studio Inc., needed a continuous pipeline to keep their machine learning models accurate and up to date as new image data comes in. Their existing process relied heavily on manual labeling, training, and deployment, which slowed down model updates and made it difficult to scale. A more automated and flexible solution was required to support timely updates and edge deployment.

7.2 What We Delivered

To address our partner's need, we developed an **automated**, **continuous**, **dynamic**, and **efficient** pipeline that streamlines labeling, human-in-the-loop review, model training, optimization, and deployment.

- **Automated**: From labeling to training and compression, everything flows with minimal manual effort.
- **Continuous**: Updated models help label data in future runs, improving over time.
- **Dynamic**: Easy to plug in new models or modules as requirements change.
- **Efficient**: Greatly reduces time across key stages including labeling, fine-tuning, and compression, resulting in faster end-to-end model iteration cycles.

7.3 Limitations & Recommendations

While our pipeline meets the core requirements, there are some limitations that restrict the scalability of the pipeline:

- **Local-first design**: Running the pipeline locally limits deployment flexibility.
- **Scattered data and model files**: Data and model files are saved in local folders, which makes tracking progress and maintaining consistency more difficult.
- **No monitoring interface**: Without a monitoring interface, it's hard to observe pipeline behavior in real time.

Hence, we propose the following three recommendations for future development:

- **Add cloud support**: Integrating cloud storage and remote triggering to allow the pipeline to run in more flexible environments.
- **Integrate a lightweight database**: Using a small database (e.g., SQLite) to store metadata for images, labels, and model versions would improve organization, enable easier tracking, and support more complex queries and analysis. As an additional task, we have designed a simplified and scalable database schema to support the core components of our wildfire detection pipeline. The structure is designed to be future-proof and manageable by the partner team.
- **Develop a simple monitoring dashboard**: A lightweight dashboard would help users monitor pipeline runs, track model status, and show performance in real time.

Limitation	**Recommendation**
Local-first design	Add cloud storage and remote triggering for better scalability
Scattered data and model files	Integrate a lightweight database to centralize images, labels, and model metadata

Limitation	Recommendation
No monitoring interface	Build a simple dashboard to track pipeline runs and model performance in real time

References

Defence Research and Development Canada. 2022. "DRDC Tests Early Detection Wildfire Sensors." https://science.gc.ca/site/science/en/blogs/defence-and-security-science/drdc-tests-early-detection-wildfire-sensors.

Heartex. 2021. "Label Studio: Data Labeling Platform." https://labelstud.io/.

Jocher, Glenn, Ayush Chaurasia, and Jing Qiu. 2023. "Ultralytics YOLOv8." https://github.com/ultralytics/ultralytics.

Liu, Shilong, Zhaoyang Zhang, Yujie Lin, Tete Zhang, Chen Qian, Alan L Yuille, Xin Lu, Pengchuan Gao, and Jiuxiang Gu. 2023. "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection." *arXiv Preprint arXiv:2303.05499*. https://github.com/IDEA-Research/GroundingDINO.